智能运维平台

智能运维平台产品技术方案

领导汇报版 技术组件与运行逻辑

文档版本 V6.1

发布日期 2026 年 11 月 1 日

适用范围 非虚拟化智能运维

以统一资源关系和可核验事实为基础，融合专业规则与大模型调查，贯通问题分析、人工处置和恢复验证。

本方案说明产品使用的技术组件、组件之间的数据与调用关系，以及自然语言交互、智能调查和运维闭环的核心机制，展示技术体系与建设价值的对应关系。

汇报主线

第 1 至 3 章 产品结论 总体架构 核心组件分工

第 4 至 12 章 观测与关系 执行分流 事实与判定 证据校验

第 13 至 18 章 持久运行 安全控制 处置验证 部署治理

第 19 至 20 章 典型场景推演 核心技术优势与建设价值

01 汇报摘要

智能运维平台围绕 Kubernetes 运行环境，将资源状态、指标、日志、通信和硬件观测组织为统一的运维事实，通过自然语言驱动只读调查，以固定取证、规则判断和证据校验支撑分析，再衔接操作者本人确认的处置与恢复验证。产品建设的核心是把数据、专业诊断、模型分析和执行责任纳入同一技术体系。

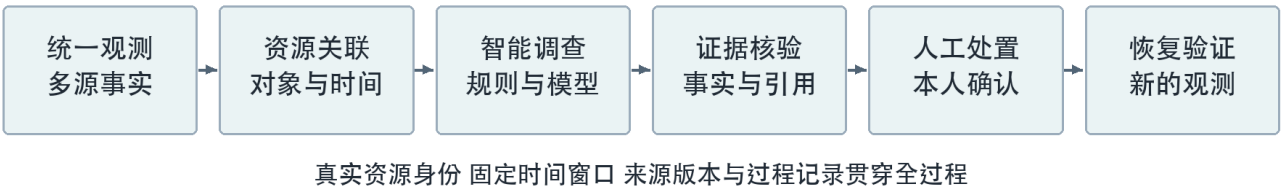


图 1 智能运维从观测到恢复验证的总体链路

技术体系的总体结论

平台采用“成熟组件复用与平台业务控制相结合”的架构。VictoriaMetrics、VictoriaLogs 和 DeepFlow 分别提供指标、日志和网络观测；Ariadne 与 Kubernetes Ontology 组织资源及依赖关系；K8sGPT 原生 Analyzer、Node Problem Detector 规则及版本化诊断配方核验异常；HolmesGPT 通过 MCP 组织自然语言调查，指定故障链任务按固定序列完成取证与有限规则评估。

Go 业务服务掌握任务、权限、预算、查询编译和结果提交；PostgreSQL 保存业务过程，SeaweedFS 归档证据；OPA 执行策略判断，OpenBao 管理服务证书与密钥；Command Runner 在隔离任务中执行本人确认的实际命令。各组件围绕真实对象、固定时间、来源版本和引用标识协同。

产品核心技术优势

第一，统一资源身份与关系，使应用、节点、存储、通信和服务器观测在同一对象体系中关联。第二，将自然语言编译为受控查询和有记录的工具调用，使分析依托实际读取结果。第三，以版本化规则、引用校验和执行后观测贯通诊断与验证，使判断依据和处理责任能够复核。

建设价值

平台沉淀统一的数据口径、诊断规则、调查过程和处置记录。管理层能够从同一事实链查看问题对象、检查依据、分析版本、实际操作及恢复结果；专业经验以规则和工具契约持续积累，模型能力在同一治理体系内接入和更新。

02 总体技术架构

平台按交互、业务控制、调查分析、观测来源和数据保障划分技术职责。独立进程承担运行隔离，内嵌库承担确定性计算，平台适配器统一外部来源。数据读取沿任务授权执行，持久提交由业务服务控制。

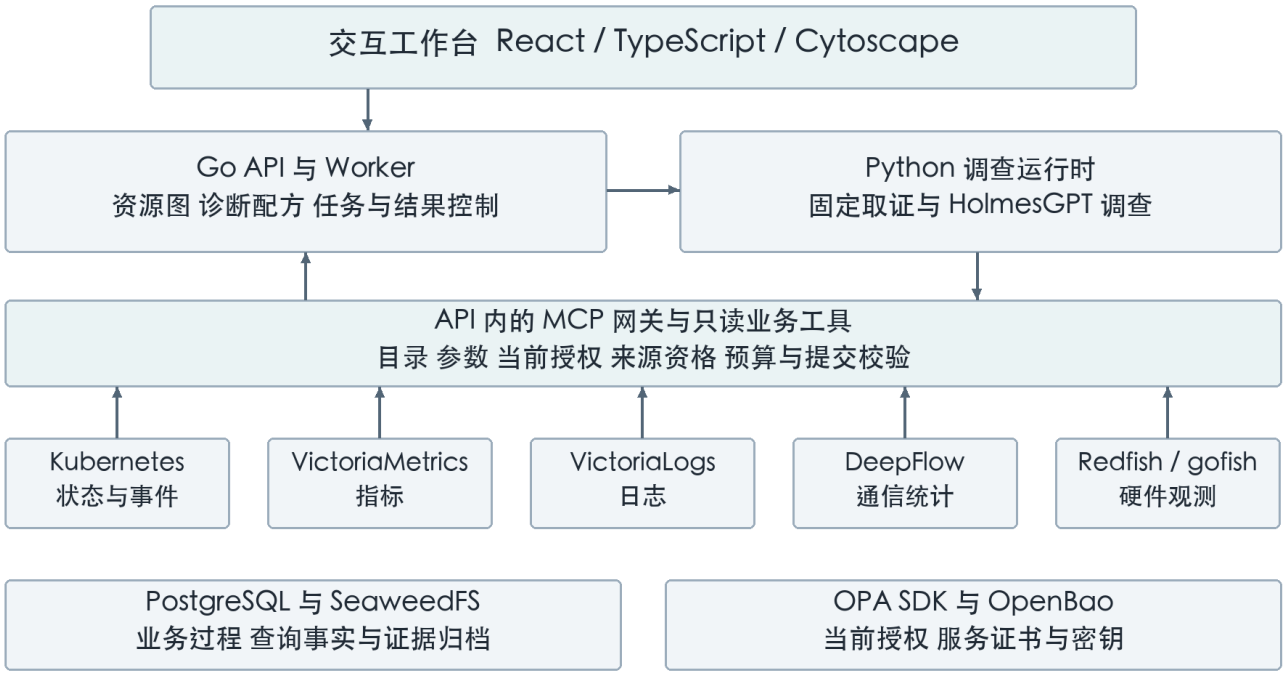


图 2 核心组件及业务调用与观测接入架构

业务控制与智能调查分工

React 与 TypeScript 构建统一工作台；Go API 接收任务并校验身份、对象和范围，Go Worker 推进后台任务。Python 调查运行时分流固定取证与 HolmesGPT 模型调查。模型调查向登记的模型服务提交问题、已知事实和工具说明，由模型提出检查请求，MCP 网关交给受控业务工具执行并返回结果；固定取证按规定序列调用同一工具链。

事实流与操作流贯通

资源图、指标、日志、网络及硬件适配器提供只读事实；平台保存步骤和引用，校验结构化分析。实际修改进入本人确认的 Command Runner 路径，执行完成后重新读取观测，形成独立恢复报告。PostgreSQL、SeaweedFS、OPA 和 OpenBao 分别支撑过程存储、证据归档、策略和凭据管理。

03 核心组件与部署分工

产品以 Go 模块化单体组织业务域，API 与 Worker 作为独立进程运行；调查运行时和命令执行任务按职责隔离。资源图、策略引擎和规则适配集成在平台内部，组件部署与业务语义保持清晰对应。

核心组件	集成方式	在产品中的责任
React 与 TypeScript	Web 工作台	任务交互、事实引用、调查进度和处置入口
Go API 与 Worker	独立进程共享业务模块	授权、任务调度、查询编译、步骤和结果提交
Python 调查运行时	独立进程集成 HolmesGPT	固定取证、模型调查及结构化结果提交
Ariadne 与 Ontology	Go 内嵌图与语义适配	资源关系、依赖查询、影响范围和诊断路径
PostgreSQL	数据库服务	会话、任务、步骤、正式分析、审计与业务事务
SeaweedFS	S3 兼容归档服务	证据片段、执行输出及内容摘要对应的归档对象
OPA 与 OpenBao	内嵌策略引擎及凭据服务	业务授权与服务证书、密钥、短期凭据管理
Command Runner	Kubernetes 一次性 Job	执行本人确认的命令，回传输出与执行状态

组件组合的工程依据

API 与 Worker 共用业务模块和事务规则，使权限、状态机及引用校验保持一致。Python 运行时集成固定取证与模型调查，业务控制留在 Go 服务；数据库保存可接管的任务状态，使调查执行与页面连接分别管理。该分工把模型调用、业务一致性与执行隔离落实到具体进程和组件。

协议和框架承担明确职责

MCP 是工具调用协议，JSON Schema 约束输入输出结构，OIDC 提供身份接入协议，SSE 向页面传递持久事件进度。它们分别连接组件，业务授权、诊断规则和证据判定由对应平台模块执行。

04 观测组件与数据接入体系

观测层按数据类型接入事实，业务层统一对象、时间和来源语义。平台读取已登记的后端，通过固定模板和范围映射形成查询结果；集群当前状态由 Kubernetes API 读取，时序记录由相应观测后端提供。

数据组件	提供的数据	平台处理方式
Kubernetes API	资源状态、事件及受限容器日志	投影资源字段，绑定 UID、集群和命名空间
VictoriaMetrics	指标时序及聚合结果	按固定模板查询，计算趋势、比较及数值事实
VictoriaLogs	集中日志及节点内核日志记录	按对象与窗口检索，保留样例和来源语义
DeepFlow	通信关系与 L4/L7 统计	绑定通信方向、观测点、实体映射和查询窗口
Redfish 与 gofish	服务器资产、部件及健康观测	按登记的 BMC 读取，映射节点和硬件实体

接入过程形成统一事实

来源注册保存实际后端、认证引用、范围映射与版本；查询同时绑定数据集、操作、模板、资格证明和时间窗口。返回内容经过字段投影、脱敏和大小限制，数值由服务端计算，单位与实际覆盖随结果保存。

metrics-server 作为 Kubernetes Resource Metrics 的发现和接入组件，承担资源指标接口职责；vmalert 的只读探测核对规则列表能力。历史指标分析依托 VictoriaMetrics，日志历史依托 VictoriaLogs，各能力按实际来源登记开放。

数据接入与采集职责分开管理

VictoriaMetrics 与 VictoriaLogs 的部署配置管理存储和查询端，采集接入由现场观测链路及来源注册绑定；平台在查询前核验实际覆盖。DeepFlow 通过专用 Profile 和适配器接入通信观测。结果明确编码总体统计、有限样例、无数据和零值，保持数据口径一致。

05 资源关系图与跨层关联技术

资源关系图为智能运维提供共同对象基础。Ariadne 负责资源图快照和关系解析，Kubernetes Ontology 语义适配将资源投影为统一节点及关系；平台在此基础上增加身份、范围、时间和硬件映射约束。

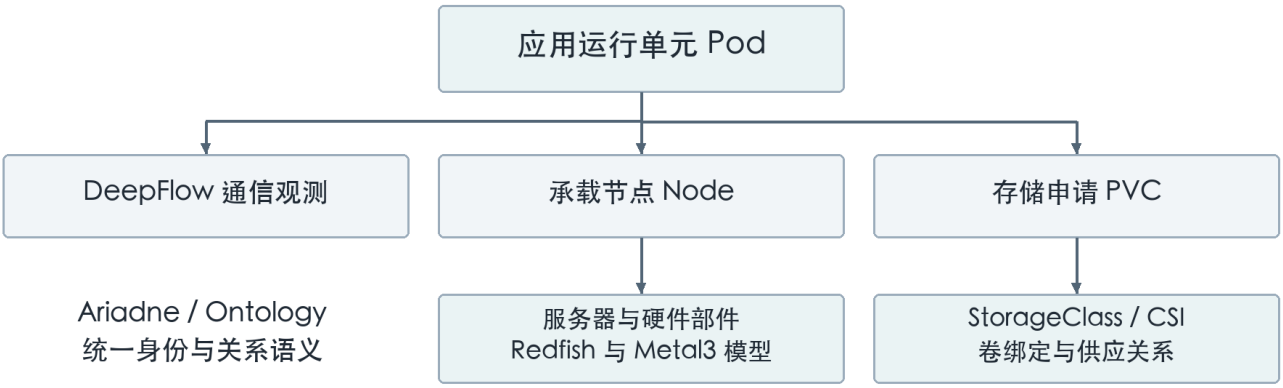


图 3 统一资源图连接应用 节点 存储 网络与硬件

稳定身份保证关联准确

Canonical ID 编码资源域、租户、所属范围、API 组、资源类型和稳定标识。Kubernetes 资源使用实际 UID；同名资源重建后形成新的身份，原有证据仍绑定原对象。关系解析核对 OwnerReference、显式引用、标签选择器、调度与存储绑定，并复核引用中的 UID。

图语义组织调查方向

平台区分调度、挂载、卷绑定、存储类型、CSI 供应、组件归属和硬件承载等关系。查询按关系方向、类型、深度和规模执行，并逐项核对当前授权。关系来源、观测时点、有效期和版本随结果保留，历史观测与当前快照保持各自时间语义。

硬件和可视化使用同一对象体系

gofish 作为 Redfish 客户端读取登记的 BMC；平台复用 Metal3 HardwareDetails 数据模型组织 CPU、内存、存储及厂商信息，经可信节点映射投影到同一资源图。Web 使用 Cytoscape 展示授权后的拓扑。图查询和页面展示由此围绕相同实体与关系展开，图能力内嵌在 Go 服务中。

06 原生检查组件与专业诊断规则

智能运维的专业判断由原生检查和版本化诊断配方共同支撑。检查组件读取确定的数据并输出异常事实，配方核对对象、路径、条件与证据；模型使用这些结果组织调查和解释。

检查组件	使用方式	产出的诊断依据
K8sGPT 原生 Analyzer	固定参数执行 Pod、Node 和 PVC 只读检查	原生异常条目及对应资源
Node Problem Detector	复用锁定的内核日志规则，对证据记录重放	内核异常命中、规则版本及原始证据引用
Coroot Check	复用确定性阈值计算逻辑	数值与阈值比较后的检查状态
平台诊断配方	按版本执行事实条件与依赖路径核验	支持、反证、补证项和正式评估记录

原生检查形成标准化输入

K8sGPT 的适配器采用固定资源过滤和 JSON 输出，校验锁定的二进制及结果结构，执行原生 Analyzer 检查。NPD 规则针对带对象、时间和版本的内核日志证据逐条匹配，保留原始记录边界。Coroot Check 将有效数值与确定阈值比较，缺失或非法输入编码为相应检查状态。

配方将专业条件固化为可核验规则

配方定义入口对象、图查询路径、必要事实、确认与排除条件、工具清单、预算和恢复检查。内存部件、节点及存储供应使用各自配方，输入绑定 Finding、图来源、证据及版本。重复派生事实按独立证据组去重，事实冲突、覆盖和时效进入评估结果。

复用价值沉淀在平台规则体系

原生检查提供专业事实，平台完成对象归属、证据规范化、配方编排和正式评估。NPD 以规则复用集成，Coroot 以计算逻辑复用集成，Metal3 以硬件模型复用集成，各自承担明确的技术职责。

07 自然语言任务与执行分流

平台先按页面任务类型、绑定对象和已提交的任务请求确定处理路径，再组织模型与工具。问数、模型调查和指定故障链分别采用相应机制，三条路径共用任务状态、权限、工具账本与事实引用。

资源名称在授权范围内解析到真实 UID，多义目标经持久澄清确定；“它”等指代由平台已验证的实体引用解析。对象、范围与时间保存到本轮任务，后续调用沿该绑定执行。

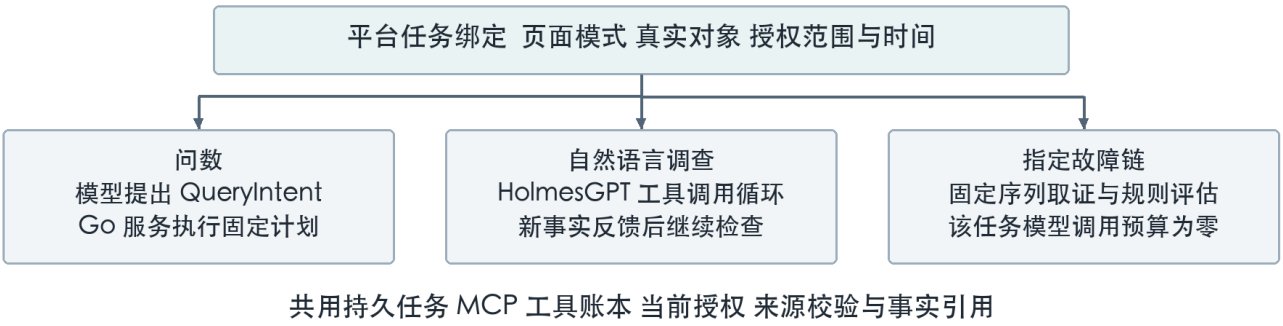


图 4 问数 模型调查与指定故障链的执行分流

问数将问题转成受控计算

例如“按健康状态统计 Pod”，模型读取数据集定义并提出结构化 QueryIntent，Go 服务校验后完成计数，回答引用保存的数值事实。数据集、算子、范围和计算方式由平台目录及查询编译器约束。

模型调查根据新事实选择后续检查

自然语言排障与追问使用 HolmesGPT 的工具调用机制。平台先读取任务上下文及规定的必要事实，再把工具目录和已提交结果提供给模型；模型提出下一项检查，平台校验执行，提交的新事实返回模型，形成调查反馈循环。

指定故障链按固定次序取证

提交明确的故障链请求后，服务端将该任务设为 evidence_only，模型调用及输入输出 Token 预算均置为零。Python 运行时执行固定取证序列，经同一 MCP 工具链读取对象历史、异常、变化和适用关系，再调用服务端有限规则评估。该任务按固定规则形成观察和缺口，后续自然语言解释使用已核验的上下文。

08 问数意图如何形成实际计算

问数的核心是将语言含义落实为目录允许的计算。以“按健康状态统计 Pod”为例，模型表达统计意图，服务端绑定实际范围、执行聚合并保存结果；自然语言回答继续引用该份结果。

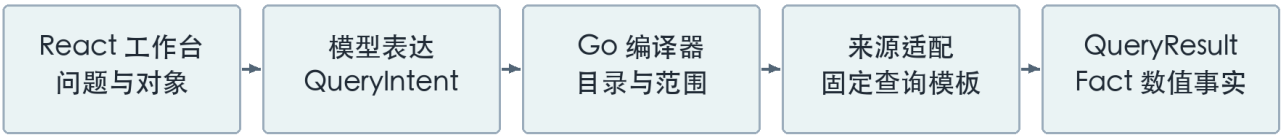


图 5 自然语言形成受控查询与服务端事实

从一句话得到确定的意图字段

数据集使用 resources.current，操作为 aggregate，指标为 resource_count，分组维度为 health，过滤条件指定 kind 等于 Pod。模型通过 describe_analysis_dataset 读取上述字段及语义，再将 QueryIntent 交给 execute_analysis_query。当前资源数据集采用当前授权快照口径。

从意图字段生成固定计划

Go 编译器先核对 JSON Schema、数据集、算子、维度和必要过滤条件，再将当前授权与页面选择组合为有效范围。编译后的计划固定查询方式、目录版本和范围摘要，资源图在完整授权范围内先筛选再聚合；指标、日志与网络数据集采用各自的固定来源模板。

任务主体、租户及授权范围来自平台绑定。相对时间以服务端请求时刻和会话时区解析，历史数据集按绝对窗口执行；当前快照与历史时序使用各自目录口径。每次读取都保留实际来源及覆盖状态。

从计算结果得到可解释答案

服务端保存 QueryResult，并为数值单元生成 Fact 标识，记录单位、统计口径和来源。模型提交对结果的解释及这些引用，展示层从保存的结果读取数值。新一轮询问当前数量时重新查询，历史回答仍对应原有快照。

该链路把语言理解、授权、计算与解释衔接起来，统计数值由服务端产生，模型负责说明实际分布及其含义。

09 调查取证如何逐步推进

模型调查将“当前已知事实”和“仍需检查的事项”持续反馈到下一轮工具选择。HolmesGPT 组织模型调用循环，MCP 连接平台只读工具，Go 服务控制真实执行和结果提交；指定故障链另走固定取证序列。

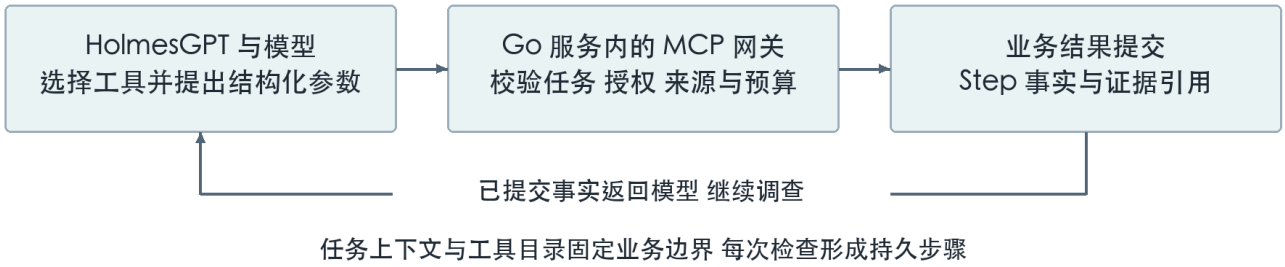


图 6 HolmesGPT 与 MCP 协同完成多步取证

平台先执行规定的上下文读取

Worker 领取任务并签发上下文，运行时先经 MCP 提交 get_assistant_context。故障追问按任务读取正式引用，带相应调查请求时预读目标历史、异常和变化。这些必要读取由程序组织，已完成结果写入 Step 后再进入模型上下文。

模型根据缺少的信息提出下一项检查

模型取得当前问题、已验证对象、工具说明和实际返回内容，再提出工具名称与参数。例如读取历史使用 resource_history，参数为目标 canonicalId 与 windowRole=fault；实际绝对窗口从服务端任务解析。检查内容与参数由当前工具目录约束，模型收到新事实后选择后续工具或提交分析。

每项调用先分配 Step，网关核对服务身份、签名任务、租约、参数、授权、来源和预算，再读取实际后端。返回内容经脱敏和输出契约校验后提交；已提交的相同读取按任务与参数摘要复用，随后作为工具结果反馈给模型。

状态与预算决定执行何时结束

模型提交结构化结果后，平台执行引用和归属校验；步骤、模型调用、Token、时间及结果大小均受预算限制。超时、取消、调用失败和预算耗尽保存对应状态及已完成步骤，回答保留实际覆盖和未完成检查。指定故障链按固定序列推进和停止，其后续检查按明确的缺口与工具映射发起。

10 事实条件如何形成规则判断

规则判断将具体观测与事先定义的条件逐项比对。以 pvc-csi-failure/v2 正式配方为例，候选类型为 CSI 供应组件故障，确认条件包含 PVC 状态、CSI 错误及真实供应路径。以下说明该配方的条件关系。

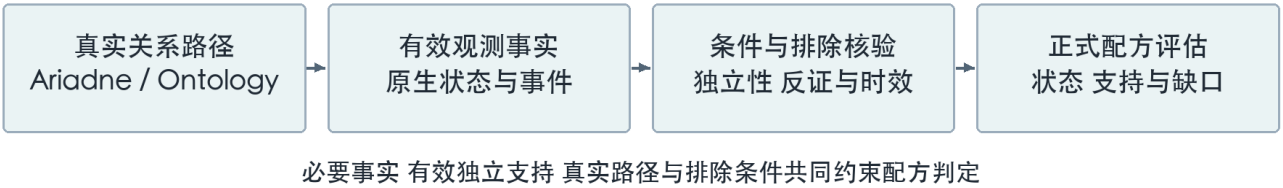


图 7 正式配方核验真实路径 有效事实与排除条件

判断条件	读取的具体事实	核验方式
pvc-pending	目标 PVC 的 phase 为 Pending	稳定身份与目标一致，观测位于有效窗口
csi-error	供应失败、挂载或附加错误事件	事件关联目标，报告控制器与图中的 CSI 驱动身份对应
storage-path	Pod 挂载 PVC，PVC 使用 StorageClass，后者关联 CSIDriver	三段真实关系、端点、来源和时间均符合配方路径

有效事实先经过来源与时间核验

正式引擎核对证据摘要、已验证归档、时间可靠性与窗口；该配方的必要证据时效为 300 秒。原始证据按独立证据组去重，派生副本保留来源追踪，确认条件使用有效的独立事实。图数据同时核对完整性、新鲜度和来源。

必要条件和排除条件共同约束判断

只有必需事实齐备、图和来源满足条件、无反证与冲突，并达到至少两组有效独立支持时，该配方评估才形成 confirmed。PVC 已为 Bound 的事实进入反证；控制器身份不匹配时 CSI 错误条件不成立；缺少真实路径时保留 storage-path 缺口。

图关系落实检查对象与依赖路径，事实条件落实该配方中的诊断依据。正式结果连同支持、反证、缺口、配方版本和输入摘要保存，再由提交流程复核实际依赖与归档内容。

11 模型分析与正式根因的判定边界

平台将调查观察、模型解释和正式根因分析（RCA）版本分别管理。各类结果使用不同的输入与提交条件，管理层能够沿同一问题查看事实、研判及正式记录的关系。

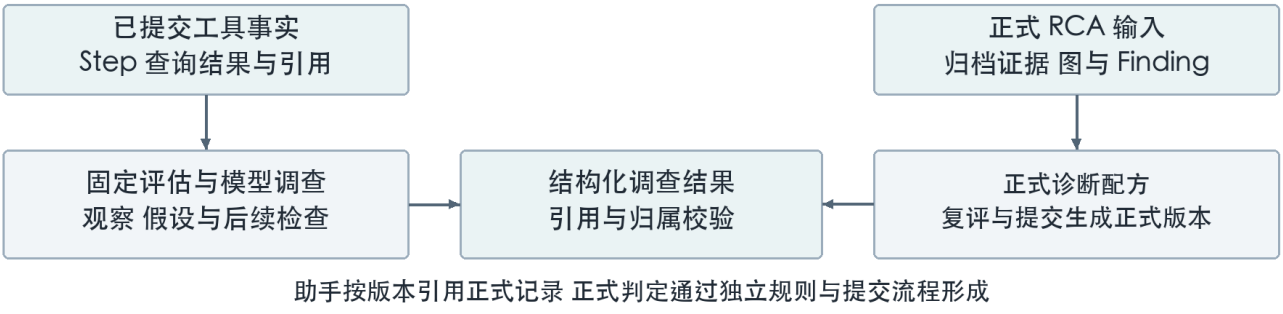


图 8 调查分析与正式根因分别形成并按引用关联

结果类型	形成机制	结果的技术含义
资源与故障链评估	服务端有限规则读取成功提交的工具 Step	记录观察到的支持、冲突、覆盖和待检查项
模型调查分析	HolmesGPT 将工具事实反馈给模型，再校验结构与引用	解释现象，组织诊断假设及后续检查
正式 RCA 版本	适用的诊断配方及独立提交流程	按具体规则条件保存正式判定、输入及版本

引用校验落实依据的真实性与归属

平台核对引用是否来自已成功提交的实际记录，是否属于当前对象、范围和版本，数值是否对应服务端计算结果。引用校验保障依据可追溯；专业判断由适用规则及事实条件承担。模型的解释保留假设、支持、反证和补证项。

正式确认通过单独的提交门槛

助手 evaluate_fault_chain 工具评估观察和关系，不能确认正式 RCA。正式提交重新执行配方评估；confirmed 结果还需复查实际资源图版本、归档内容摘要、Finding 版本、来源和授权，保存追加的正式版本。助手回答与正式判定由各自流程形成，再通过明确引用关联。

结果状态对应实际完成的工作

工具成功表示本次读取完成，来源覆盖说明取到哪些数据，模型分析说明研判，正式根因状态说明配方判定。各状态分别编码和展示，后续取证形成新的步骤与分析版本，既有结果继续保留原有时点和依据。

12 PostgreSQL 与 SeaweedFS 证据体系

平台将结构化业务过程与证据内容分开保存，再以标识、摘要和版本建立关联。PostgreSQL 保存会话、任务、步骤、查询事实、正式分析和审计记录；SeaweedFS 通过 S3 接口归档选定的证据片段、执行输出及相关材料。

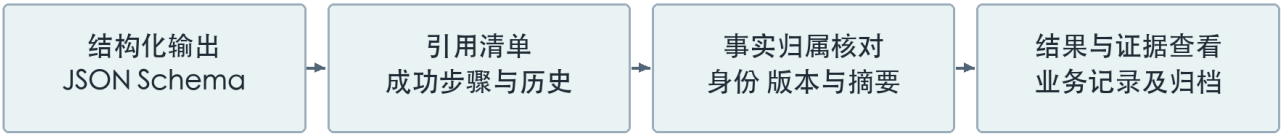


图 9 分析引用与业务记录及证据归档对应

引用对象	定位的记录	提交时的核对内容
QueryResult 与 Fact	查询结果与数值单元	计划摘要、结果归属、行列位置及实际计算值
Evidence	证据元数据及归档内容	资源、来源、窗口、数据等级、摘要与保留状态
Entity 与 Relation	真实对象及资源关系	当前范围、关系端点、类型和来源摘要
正式分析与处置计划	指定故障的实际版本	故障归属、内容摘要和版本对应关系

模型输出与事实记录绑定

分析按 JSON Schema 提交，业务服务汇集本轮成功步骤和重新授权的历史引用，逐项核对声明的依据。查询数值定位到服务端结果，关系定位到已提交关系，正式证据定位到元数据与归档对象。工作台据此打开实际步骤、事实和证据详情。

归档服务支撑业务复核

平台在归档流程管理内容摘要、加密、保留策略和 Legal Hold，业务记录保存对象引用及状态。指标和日志仍由各自后端提供查询，归档保存调查及处置所需内容。数据来源、时间与分析版本共同构成复核依据。

13 Go Worker 与持久任务机制

智能调查通过持久任务运行。Go API 接收提问并保存轮次与 Job，Worker 从持久队列领取任务，PostgreSQL 记录执行状态和每次工具 Step；页面通过持久事件和 SSE 呈现进度。

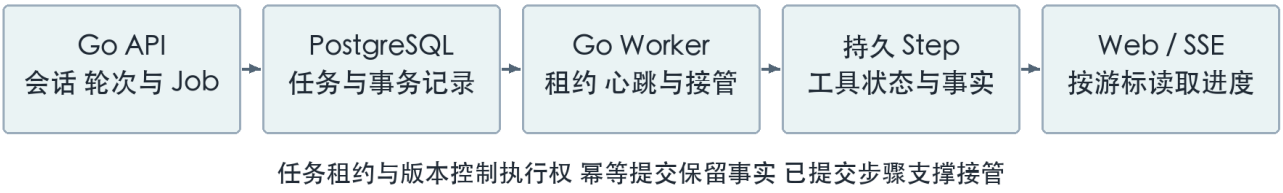


图 10 Go 与 PostgreSQL 管理持久任务和执行过程

租约和版本管理执行权

Worker 领取任务时取得有效期与租约版本，通过心跳维持执行权。签名上下文和提交请求绑定当前租约，写入时再次核验。接管任务使用新版本，迟到的旧执行结果按租约检查拒绝，已成功提交的读取继续作为接管依据。

幂等与预算管理执行过程

任务提交使用幂等键和会话修订号，工具调用按名称与参数摘要分配步骤。调用预算的预留、开始、完成与失败分别记账；时间、Token、步骤、证据量和结果大小共同约束调查。任务终止原因与已完成事实分开保留，便于继续处理和过程复核。

连续追问保持对象与事实一致

会话保存范围，轮次保存提问及时间，Step 保存实际读取，分析保存引用与版本。历史进入新轮次前复核当前权限，保留原观测时点；新查询产生新的事实记录。页面重连凭事件游标读取进度，调查生命周期由后台状态持续管理。

该机制把一次对话形成的检查过程转化为可交接、可恢复和可审计的业务记录，为持续排障提供技术基础。

14 OPA 与 OpenBao 安全控制

平台将人员身份、工作负载身份和业务授权分别落实到相应组件。人员按平台内置账号或 OIDC 模式接入，Keycloak 承担 OIDC 身份接入；OpenBao 提供服务证书、密钥和短期凭据；OPA SDK 内嵌于 Go 服务，执行 Rego 策略判断。

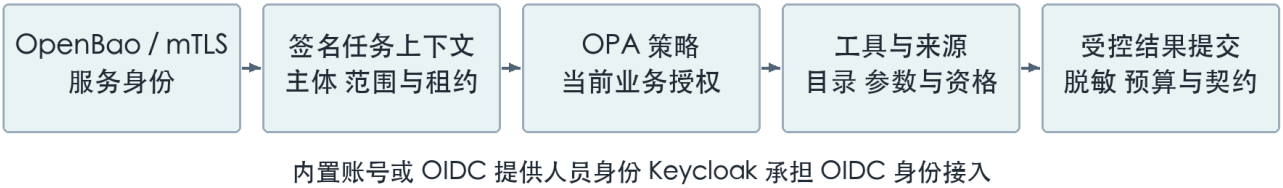


图 11 人员身份 服务身份与当前授权协同控制

身份与具体任务绑定

调查运行时与 MCP 网关通过 mTLS 验证服务身份，签名任务上下文绑定任务、主体、租户、范围、工具目录、有效期和租约。平台核对实际调用服务与上下文，再读取任务状态和当前执行权。OpenBao 的 PKI、Transit 和 SSH 签发能力分别支撑证书、签名加密与短期访问凭据。

授权贯穿读取与提交

OPA 根据当前主体、角色、目标和操作执行策略判断。平台在任务、工具参数、来源映射、历史引用、证据读取及事件披露处持续核对当前授权；工具输出经过脱敏、数据等级和结构校验。日志及用户文本作为输入数据处理，任务授权来自平台建立的上下文。

角色与责任形成管理闭环

运维操作员与平台管理员按各自角色和范围授权。调查使用只读工具，实际修改由操作者本人输入并确认命令；高风险内容与目标、配置及身份增强验证绑定。审计记录保留责任主体、实际内容、执行事件和关联分析，使技术控制与操作责任对应。

15 Command Runner 与隔离执行

处置以操作者本人确认的实际命令为依据。Go 业务服务固定目标、命令、执行配置和风险确认，Worker 按获准路径派发 Command Runner；Runner 以一次性 Kubernetes Job 运行，回传输出和执行状态。

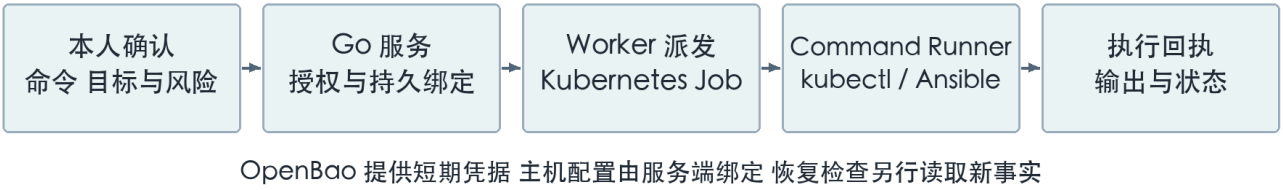


图 12 实际命令通过隔离 Job 和受控通道执行

Kubernetes 通道使用限定执行环境

Runner 镜像包含平台执行器、kubectl、Bash 和锁定的运行依赖。Kubernetes 通道使用服务端绑定的目标与凭据，在隔离 Job 中执行本人确认的命令；执行记录、目标身份和配置共同控制派发，输出按对应执行标识保存。

主机通道复用 Ansible Runner 传输

主机通道由平台生成传输配置，复用 Ansible Runner 与 ansible-core 执行确认的 Bash 命令。OpenBao 签发短期 SSH 凭据，目标与已知主机信息由服务端绑定。Ansible 在该链路承担受控传输与执行职责，命令内容及操作责任由本人确认流程固定。

执行记录保持内容一致

命令、目标或执行配置发生变化时，重新执行相应确认和校验。持久记录关联操作者、实际命令、风险、执行事件、输出与回执；执行后的检查使用新的观测和独立判定。调查建议、实际操作和恢复结果分别记录，再通过共同目标及执行标识贯通。

该组件组合把智能分析落实为有边界、有责任主体和有过程记录的运维处置。

16 执行后观测与恢复验证

恢复验证将操作完成时间作为新观测边界，围绕同一目标重新读取对象与关联状态。执行回执记录命令过程，恢复配方按新的事实独立判断，结果关联到同一执行记录。

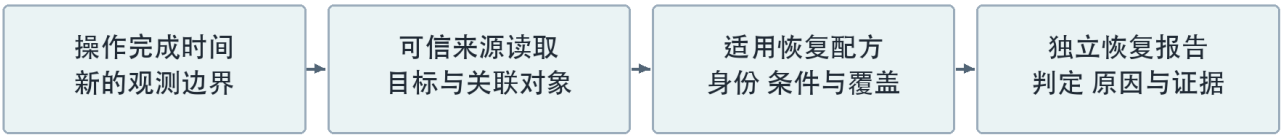


图 13 执行后新观测形成独立恢复报告

PVC 恢复按关联健康条件验证

pvc-csi-failure/v2 使用 native-related-health/v1 检查。首先核对 PVC 已为 Bound，再核对其实际 PV 名称、PV 的 Bound 状态及 claimRef 中的 PVC UID、名称和命名空间。随后核对 PV 指定的 CSI 驱动，读取消费该 PVC 的 Pod，检查实际卷挂载与 Pod 健康。

当驱动声明需要附加卷时，继续核对 VolumeAttachment 的 PV、节点、驱动和 attached 状态及错误字段。执行完成后新出现的相关 Warning 事件进入异常记录并参与判定。验证所需快照按预算、身份和关联关系完整取证。

判定取决于新的实际观测

有效观测满足对应恢复条件时记录 resolved；PVC 仍为 Pending 或 Lost、关联消费状态未恢复等情况记录 not_resolved；必要对象缺少、身份不符或证据窗口等校验不满足时记录 inconclusive。每项判定保留检查原因、实际覆盖和证据引用。

管理记录形成完整闭环

新证据核对来源、摘要、授权及操作完成后的观测时间，再形成恢复报告。平台将起始分析、本人确认、实际命令、回执与恢复报告贯通，分别展示操作是否完成和对应问题的验证结果，为交接、复盘及下一轮处理提供依据。

17 模型与数据能力治理

平台将模型、工具、数据集、来源及诊断配方分别版本化，并在任务执行时组合核验。模型接入由 Model Profile 管理，观测接入由来源注册和操作资格管理，任务开放由实际能力、权限与预算共同确定。

Model Profile 管理实际模型能力

Profile 记录实际模型与协议、工具调用和结构化输出方式、适用任务、上下文与输出上限、超时、成本策略及评估材料，生成内容引用。业务服务、Worker 和 Python 调查运行时核对同一版本，再访问登记的模型端点。部署配置对接独立模型服务，模型权重由该服务管理。

来源资格落实到具体操作

来源登记实际后端、映射与版本。指标、日志和网络查询的资格进一步绑定数据集、算子、采集版本、证明摘要、有效期及覆盖窗口。来源或映射更新后重新核验，工具按获准能力开放；DeepFlow 的通信和 L4/L7 统计按对应来源资格启用。

治理对象	固定的内容	运行中的作用
模型与工具	模型配置、目录、参数及输出契约	对齐调查能力和实际调用
查询与来源	数据集、模板、范围、窗口和资格	保持统计口径及后端映射一致
规则与证据	配方版本、独立性、摘要及归属	保留诊断和验证的判断依据
任务与预算	租约、步骤、时间及消耗	控制执行生命周期并保存过程

数据状态进入结果解释

平台分别编码零值与无数据、样例与总体、历史与当前，保留来源和完整性状态。能力治理把这些语义带入工具、分析和展示，使模型配置变化、来源变化及规则更新都能够定位到具体版本和任务。

18 部署架构与组件版本管理

平台以 Kubernetes 和 Helm 组织部署。Web、API、Worker 与 Investigator 按职责运行，处置使用一次性 Job；PostgreSQL、SeaweedFS、OpenBao 及身份接入组件由配置管理。观测后端通过实际环境发现、部署配置与来源注册建立连接。

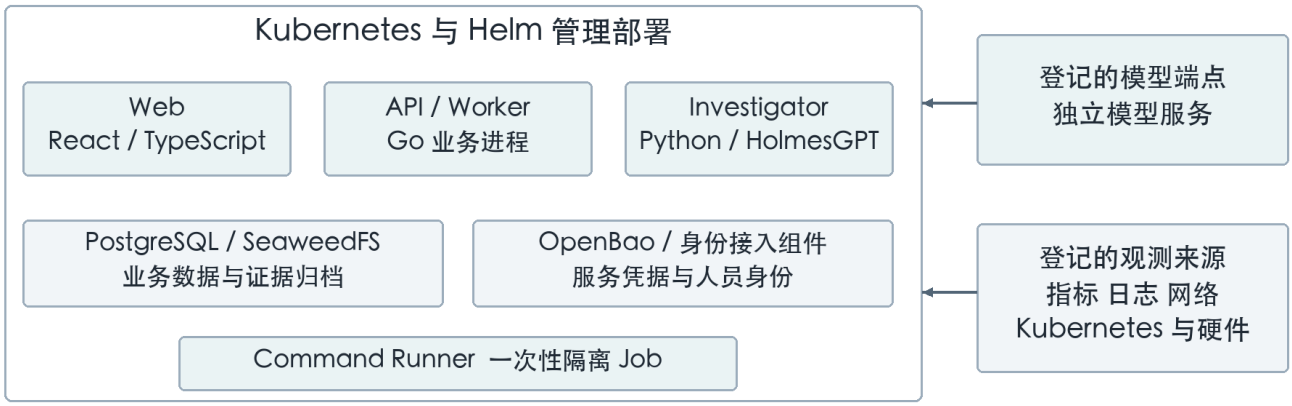


图 14 平台进程 数据服务与外部端点的部署分工

组件版本落实到交付材料

当前组件锁定配置包含 VictoriaMetrics 1.116.0、VictoriaLogs 1.52.0、HolmesGPT 0.42.0、K8sGPT 0.3.41、PostgreSQL 17.11、SeaweedFS 4.47、OpenBao 2.7.0、Keycloak 26.7.4 和 OPA SDK 1.21.0。Go、Python、Web 依赖由各自锁定文件管理，Ariadne 等内嵌依赖固定到具体版本或提交。

离线部署管理依赖与完整性

Bundle 组织镜像、Chart、二进制、Python 依赖及许可证材料，以版本和摘要绑定交付输入。安装流程按架构导入材料、应用配置并执行相应检查，运行能力按 Profile 和来源资格开放。模型端点作为独立配置接入，数据源地址与认证引用由平台登记。

部署拓扑与业务职责一致

OPA 作为 SDK 内嵌运行，Ariadne 与 Ontology 作为平台内部图和语义能力运行；NPD 规则、Coroot 计算和 Metal3 模型以复用代码集成。独立服务、一次性 Job、内嵌依赖及外部来源分别管理，部署规模与实际业务职责对应。

19 存储供应问题的技术推演

以“这个 PVC 为什么持续处于 Pending 状态”为例，平台从对象与时间绑定进入调查，逐项取得事实，再按各自流程形成观察、模型分析与正式判定。该推演说明处理逻辑和判定条件。

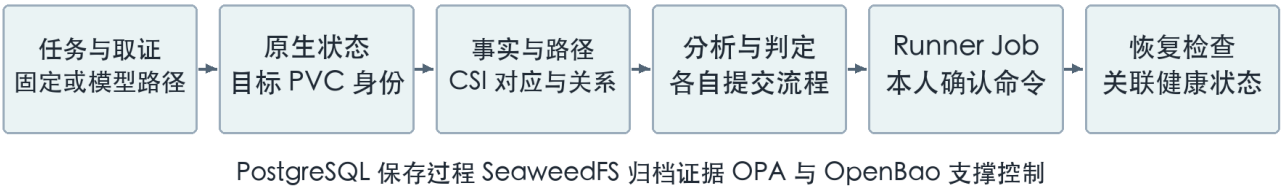


图 15 存储供应问题串联核心组件与运维闭环

第一步固定问题再执行适用取证

Go 服务确认目标 PVC 的真实 UID、Canonical ID、授权范围与窗口，保存 Job。自然语言调查先读取平台上下文及必要事实，再由 HolmesGPT 组织工具调用；指定故障链请求执行固定序列，读取目标历史、异常、变化和适用依赖。PVC 的该取证路径使用原生观测与 CSI 关系，按资源类型记录指标及日志数据集适用性。

第二步按事实解释现象与缺口

Pending 证明目标当前存在供应等待现象。真实的 Pod、PVC、StorageClass 和 CSIDriver 关系给出检查路径；供应错误事件还需将报告控制器与该驱动身份对应。工具读取形成 Step，固定评估保存支持、冲突和缺口；模型使用这些实际结果解释现象并提出后续检查。

正式 RCA 由适用配方及独立提交流程形成。必要事实、有效独立支持、图来源和时效齐备且无反证冲突时，正式引擎按配方形成确认结果；仅有 Pending 或无法对应的错误事件时，保存实际观察与缺口，继续按具体检查项补证。

第三步由人工处置进入独立验证

操作者查看依据，输入实际命令并确认目标、风险与身份。隔离 Runner Job 执行确认内容并保存回执；恢复检查读取完成后的 PVC、PV、消费 Pod、CSI 驱动及适用的卷附加状态，再记录判定与证据。资源身份、步骤标识、规则版本和引用将整条链路连接起来。

20 核心技术优势与建设价值

平台的技术优势体现在组件协同形成的运维机制：统一对象、受控查询、多步调查、规则核验、证据绑定和执行后验证。成熟组件提供专业基础，平台以业务控制和统一语义将其组织为完整产品能力。

技术优势	关键组件与机制	对应建设价值
统一资源语义	Ariadne、Ontology、Canonical ID 与来源时间	多源观测围绕同一真实对象关联
自然语言驱动	QueryIntent、查询编译、HolmesGPT 与 MCP	从人员问题直接进入可记录的取证过程
规则与模型协同	原生 Analyzer、NPD 规则及诊断配方	专业条件和分析解释保持明确依据
事实与结果对应	PostgreSQL、SeaweedFS 与引用校验	分析结论能够展开到实际步骤和证据
调查持续运行	Go Worker、租约、幂等提交及持久事件	过程可接管，历史可交接，执行可追踪
操作责任贯通	OPA、OpenBao、隔离 Runner 与恢复配方	身份、实际操作和验证结果完整关联

平台自身的核心技术形成统一控制

平台掌握资源身份投影、跨层关系适配、语义查询编译、持久调查状态机、版本化配方、引用校验和处置验证链路。模型、数据源与原生检查通过明确契约接入，系统能力由任务、事实、规则、权限及预算共同约束。

产品能力沉淀为组织技术资产

数据口径、诊断条件、调查过程、证据材料和处置记录在同一体系持续积累。领导汇报能够沿“问题对象、判断依据、实际行动、恢复结果”展开，技术评审能够继续下钻到组件、版本、工具步骤和原始事实。平台将智能分析纳入可管理、可复核的运维工作过程。